A couple of puzzling details exist in this sequence of module inclusions. The first one is this: normally, a class gains *instance methods* by including a module. But validate is a *class method* on ActiveRecord::Base. How can Base gain class methods by including modules? This is the topic of the next chapter, *Active Support's Concern Module*, where we'll also look at the metaprogramming treasure trove that hides behind this assembly of modules. For now, notice that the modules in Active Record are special. You gain both instance *and* class methods by including them.

You might also have this question: why does ActiveRecord::Base need both ActiveRecord::Validations and ActiveModel::Validations? There is a story behind these two similarly named modules: in earlier versions of Rails there was no Active Model library, and validate was indeed defined in ActiveRecord::Validations. As Active Record kept growing, its authors realized that it was doing two separate jobs. The first job was dealing with the database operations, such as saving and loading. The second job was dealing with the object model: maintaining an object's attributes, or tracking which of those attributes were valid.

At this point, the authors of Active Record decided to split the library in two separate libraries, and thus was Active Model born. While the database-related operations stayed in Active Record, the model-related ones moved to Active Model. In particular, the valid? method has its own reasons to dabble with the database (it cares whether an object has ever been saved to the database already)—so it stayed in ActiveRecord::Validations. On the contrary, validate has no relationship to the database, and it only cares about the object's attributes. So it moved to ActiveModel::Validations.

We could hunt for more method definitions, but by now you can see what Active Record's high-level design boils down to: the most important class, ActiveRecord::Base, is an assembly of modules. Each module adds instance methods (and even class methods) to the Base mix. Some modules, such as Validations, in turn include more modules, sometimes from different libraries, bringing even more methods into Base.

Before looking deeper into Active Record's structure, let's see what this unusual design can teach us.

A Lesson Learned

By including so many modules, ActiveRecord::Base ends up being a very large class. In a plain-vanilla Rails installation, Base has more than 300 instance methods and a staggering 550 class methods. ActiveRecord::Base is the ultimate *Open Class (14)*.